

RxJS Operators Deep-Dive

Part 2: withLatestFrom, startWith

August 20, 2026

TOPIC — WITHLATESTFROM

RxJS Operator | Combination Category



1. Internal Implementation

Two producer roles: source drives emission timing, others are read-only snapshots. This asymmetry is the entire mechanism.

```
type Teardown = () => void;

class WithLatestFromObservable<T, O extends any[]> {
  constructor(
    private source: { subscribe(observer: Observer<T>): Teardown },
    private others: { subscribe(observer: Observer<any>): Teardown }[]
  ) {}

  subscribe(observer: Observer<[T, ...O]>): Teardown {
    const latest: any[] = new Array(this.others.length);
    const hasValue: boolean[] = new Array(this.others.length).fill(false);

    // Passive subscriptions - just track latest value, never trigger emission
    const otherTeardowns = this.others.map((other$, i) =>
      other$.subscribe({
        next: (v) => { latest[i] = v; hasValue[i] = true; },
        error: (err) => observer.error(err),
        complete: () => {} // completion of 'other' doesn't kill withLatestFrom
      })
    );

    // Active subscription - this is the only thing that triggers next()
    const sourceTeardown = this.source.subscribe({
      next: (v) => {
        if (hasValue.every(Boolean)) {
          observer.next([v, ...latest] as [T, ...O]);
        }
        // no value yet from an 'other' -> silently drop, no buffering
      },
      error: (err) => observer.error(err),
      complete: () => observer.complete()
    });

    return () => {
      sourceTeardown();
      otherTeardowns.forEach(t => t());
    };
  }
}
```

2. Live Trace

```
const clicks$ = /* emits 'click' at t=0,3,5,8 */;
const formValue$ = /* emits form state at t=1,4,6 */;

clicks$.pipe(withLatestFrom(formValue$)).subscribe(console.log);
```


TOPIC — STARTWITH

RxJS Operator | Creation/Utility Category



1. Internal Implementation

No producer of its own for the prepended values - they're pushed synchronously inside `subscribe()`, before the real source producer even starts. Not a merge, not a race. Pure ordering trick at subscribe-time.

```
class StartWithObservable<T> {
  constructor(
    private source: { subscribe(observer: Observer<T>): Teardown },
    private initialValues: T[]
  ) {}

  subscribe(observer: Observer<T>): Teardown {
    // Synchronous emission BEFORE source subscription even happens
    for (const val of this.initialValues) {
      observer.next(val);
    }

    // Now subscribe to actual source - continues from here
    return this.source.subscribe({
      next: (v) => observer.next(v),
      error: (err) => observer.error(err),
      complete: () => observer.complete()
    });
  }
}
```

2. Live Trace

```
const search$ = /* emits 'angular' at t=2, 'rxjs' at t=5 */;

search$.pipe(startWith('')).subscribe(console.log);
```

t	Internal action	Output
0 (subscribe call)	loop emits "" synchronously	"
0 (still sync)	source.subscribe() runs, source is cold/async so nothing yet	-
2	search\$ -> 'angular'	'angular'
5	search\$ -> 'rxjs'	'rxjs'

Critical detail: the "" fires at t=0 synchronously during subscribe, not "before the stream starts" in some abstract sense - it's literally in the call stack of `.subscribe()`, before `source.subscribe()` is even invoked.

If source were itself synchronous (e.g. of('angular')), order is still: "" then 'angular' - because the loop runs first in the function body.

3. Comparison

